

Keywords: agentic reinforcement learning • policy optimization • value functions • LLM post-training

SAPO: Single-Rollout Autoregressive Policy Optimization for Agentic RL

A Shared Autoregressive Backbone Yields Policy and Value Functions at No Extra Memory Cost, Beating PPO and GRPO on Multi-Turn Agent Tasks

By Dayang Liang, Lang Feng, Bo An, Yunlong Liu

arXiv:2608.19842 • Preprint, August 2026

Reinforcement learning for multi-turn LLM agents faces a harsh trade-off: critic-based methods like PPO provide token-level credit but double memory overhead, while critic-free methods like GRPO avoid a separate model but assign reward only at sequence level and stall on straggler rollouts. SAPO escapes the dilemma by reading policy distributions and value estimates from *the same* autoregressive backbone at distinct causal boundaries, then optimizing both with a single rollout per step. On ALF-World and WebShop, SAPO outperforms PPO by +15.1 pp and GRPO by +12.1 pp while cutting per-iteration runtime by 33.2%.

AT A GLANCE

Problem: Agentic RL for LLMs requires either a costly separate critic (PPO) or coarse sequence-level credit from large rollout groups (GRPO), with neither efficiently handling long-horizon tasks.

Why it matters: LLM agent deployment is blocked by training cost; a single-rollout method that matches critic-based credit quality would substantially lower the barrier to scaling.

Method: Exploit causal boundaries in the autoregressive backbone to produce policy and value heads from shared parameters; optimize PPO objectives jointly with auxiliary on-policy SARSA for value heads.

Result: +15.1 pp over PPO and +12.1 pp over GRPO on ALFWorld/WebShop with Qwen2.5-1.5B/7B; 33.2% faster per iteration.

Evidence: Ablations show that batch-normalized GAE is critical for stability; removing either the SARSA auxiliary or the shared backbone degrades performance substantially.

Agentic reinforcement learning trains LLMs to complete multi-turn tasks — web navigation, code execution, tool use — by interacting with an environment over many steps. Reward is sparse and delayed: the agent may execute dozens of actions before receiving a signal about whether the task succeeded.

Two RL paradigms dominate this setting. **PPO** maintains a separate critic model V_ϕ that predicts future return from each intermediate state, enabling fine-grained *token-level* advantage estimates. But the critic roughly doubles memory footprint and adds a second forward-backward pass per training step. **GRPO** eliminates the critic by sampling G rollouts per prompt and using group statistics as a baseline, but credit is assigned only at sequence level, and throughput is bottlenecked by the slowest rollout in each group.

SAPO proposes a third path: derive both the policy distribution and value estimate from a *single* autoregressive forward pass over a single rollout, sharing all parameters while keeping the two estimates informationally independent.

Why Existing Approaches Fall Short

PPO: The critic doubles memory, complicating deployment on commodity hardware. Training the critic on the policy’s own token context is non-stationary — as the policy changes, the value targets shift — causing instability that practitioners address with separate learning rates, KL penalties, and warm-up schedules.

GRPO and group-relative methods: Sequence-level credit means all tokens in a response receive the same advantage signal, even though early tokens set up the trajectory and late tokens close it. This masks the contribution of individual actions, slowing learning on tasks with long chains of dependencies. The straggler problem compounds this: when $G = 8$, training stalls until all eight rollouts finish, wasting compute proportional to the gap between fastest and slowest.

Prior shared-backbone attempts have fused policy and value heads at the final layer but computed them over the same token context. This conflates the *action* decision (next token) with the *state value* prediction (return from here), conflating two informationally distinct quantities.

Core Mechanism

SAPO exploits the *causal boundary* property of autoregressive transformers. At step t in the agent’s trajectory, the residual stream at position j contains information from all tokens at positions $\leq j$ but not from positions $> j$. This means:

The Big Picture

- The **policy head**, reading the stream at the position of the last token of the current observation, sees the full observation context but no generated action tokens.
- The **value head**, reading at the position just after the first action token is generated, sees the observation and the first action token — a distinct information set.

These two heads share *all parameters* (the transformer backbone), but they are causally separated: neither head’s input is derived from the other’s output position. This makes them informationally independent despite sharing weights.

Technical Formulation

Let s_t be the observation (token sequence) at step t and a_t the action generated by the policy π_θ . The shared backbone f_θ produces logits at two causal boundaries:

$$\pi_\theta(a_t|s_t) = \text{softmax}(W_\pi \cdot f_\theta(s_t)[j_\pi])$$

$$V_\theta(s_t) = W_V \cdot f_\theta(s_t \| a_t^{(0)})[j_V]$$

where j_π is the position of the last observation token and j_V is the position after the first generated action token, and $\|$ denotes concatenation.

The total loss combines the clipped PPO objective and an auxiliary on-policy SARSA value loss:

$$\mathcal{L} = \mathcal{L}_{\text{PPO}}(\pi_\theta) + \lambda \mathcal{L}_V(\theta)$$

$$\mathcal{L}_{\text{PPO}} = -\mathbb{E}_t \left[\min \left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \hat{A}_t \right) \right]$$

$$\mathcal{L}_V = \mathbb{E}_t \left[(V_\theta(s_t) - r_t - \gamma V_{\theta_{\text{old}}}(s_{t+1}))^2 \right]$$

where $\rho_t = \pi_\theta(a_t|s_t) / \pi_{\theta_{\text{old}}}(a_t|s_t)$.

Advantages use a batch-normalized λ -return GAE:

$$\hat{A}_t = \frac{\delta_t^\lambda - \mu_B}{\sigma_B + \varepsilon}, \quad \delta_t^\lambda = \sum_{k=0}^{T-t} (\gamma \lambda)^k (r_{t+k} + \gamma V_\theta(s_{t+k+1}) - V_\theta(s_{t+k}))$$

where μ_B, σ_B are the batch mean and standard deviation of unnormalized advantage estimates.

Learning or Inference Procedure

Each training iteration proceeds as:

1. Sample one rollout per prompt from the current policy π_θ .
2. Compute value estimates at all causal boundaries using the shared backbone.
3. Compute λ -return advantages and apply batch normalization.
4. Perform K gradient steps on $\mathcal{L}$ with the PPO probability-ratio clip preventing large policy updates.

5. Update $\theta_{\text{old}} \leftarrow \theta$.

At inference time, only the policy head is used; value heads are discarded. No rollout groups, no separate critic model.

What the Guarantee Says

No formal convergence guarantee is provided. The theoretical contribution is the causal-boundary argument establishing that the two heads are informationally independent despite sharing parameters. The empirical claim is that this independence is sufficient to stabilize joint optimization of the PPO and SARSA objectives.

Experimental Findings

- **ALFWorld success rate:** SAPO (Qwen2.5-7B) achieves 78.4% vs. PPO 62.1% and GRPO 65.3%.
- **WebShop success rate:** SAPO achieves 54.7% vs. PPO 40.8% and GRPO 43.9%.
- **Average improvement:** +15.1 pp vs. PPO; +12.1 pp vs. GRPO, across model sizes and benchmarks.
- **Runtime:** 33.2% faster per iteration than PPO at matched throughput; comparable to GRPO with $G = 1$ but with much better credit assignment.
- **Memory:** No separate critic model; SAPO’s memory footprint matches an inference-only LLM deployment.

Ablations and Interpretation

Batch normalization of advantages: Removing batch normalization destabilizes training on long trajectories (>20 steps). The distribution of unnormalized advantages shifts as the policy improves, causing the PPO clip to become inactive.

Auxiliary SARSA loss: Removing $\mathcal{L}_V$ degrades performance to near-GRPO levels, confirming that the value heads must be trained, not just read passively.

Causal separation: Ablating to the same position for both heads (reading from the final observation token for both policy and value) reduces performance by 4–6 pp, confirming that informational separation is necessary for independent credit assignment.

Model size: The 1.5B model benefits more from SAPO than the 7B model in absolute terms, suggesting that smaller models have more to gain from precise credit assignment.

Reference

Dayang Liang, Lang Feng, Bo An, Yunlong Liu. **SAPO: Single-Rollout Autoregressive Policy Optimization for Agentic Reinforcement Learning.**

arXiv:2608.19842, August 2026. [https://arxiv.org/
abs/2608.19842](https://arxiv.org/abs/2608.19842)